

19 | 基础平台篇：回顾与总结

2019-06-18 许式伟

许式伟的架构课

[进入课程 >](#)



讲述：姚迪迈

时长 10:10 大小 9.32M

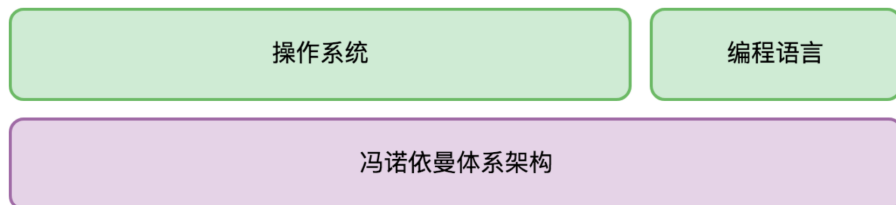


你好，我是七牛云许式伟。

到今天为止，我们第一章“基础平台篇”就要结束了。今天，让我们对整章的内容做一个回顾与总结。

抽象信息世界的骨架

基础平台篇主要涉及的内容如下。



这些内容如果展开来讲，每一系统（或模块）都会是很厚的一本书。我们的目的，当然不是为了取代这里每一个领域知识相关的专业书籍。

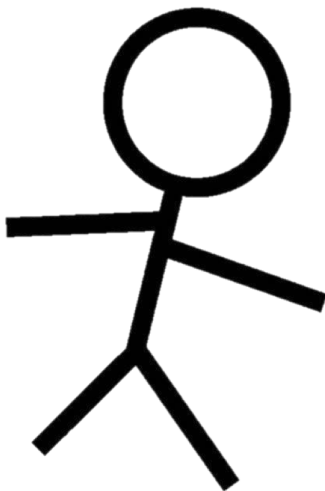
我们的核心目标是以架构为导向，抽象出系统的骨架，融会贯通，把这些领域知识串起来，拼出完整的信息世界的版图。

抽象出系统骨架的过程时信息必然是有损的，怎么才能做到忽略掉众多的实现细节，把系统以简洁易于理解的方式呈现出来？

这很大程度取决于你对系统的理解程度和抽象能力。如果我们把系统想象成一个人，大部分情况下我们比较容易对其进行详尽而具体的描述，好比下图。



这相对容易。因为你只需要陈述你看到的事实，而不必拷问背后的原因。但实际上为了在最短的时间里让别人理解你的想法，你也许应该这样来描述它，见下图。



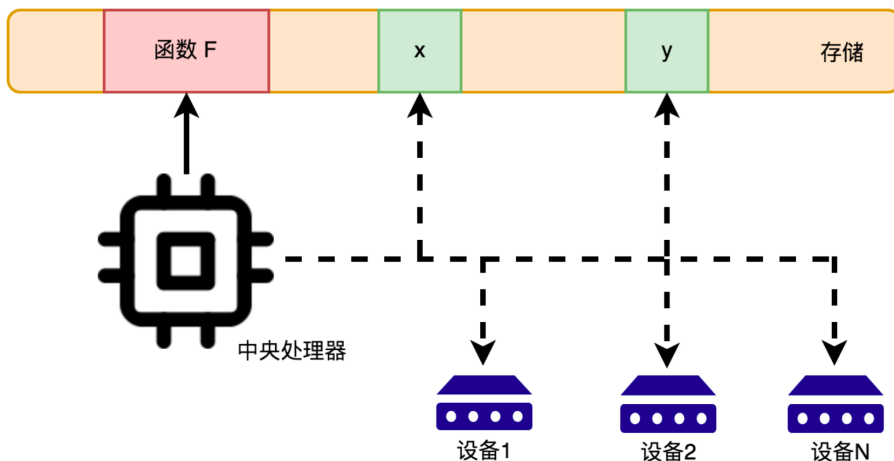
当你不是在描述这个系统本身，而是描述它与其他系统的相互关系时，你可能需要进一步简化它，变成如下图这样。



抽象有助于记忆，因为骨架需要逻辑的自治。

这种抽象能力之所以重要，是因为它是融会贯通、疏通整个信息世界的知识脉络的关键。当你做到对世界的认知可宏观、可微观，自然一切皆在掌握。

比如，本章我们首先介绍的是冯·诺依曼体系结构，我们把它抽象为“**中央处理器（CPU） + 存储 + 一系列的输入输出设备**”，并给出了系统的示意图如下。



这个图相当笼统，并没有涉及中央处理器（CPU）指令设计的真正细节。比如，我们没有介绍栈（stack）这个概念，虽然它实际上也非常关键。

为什么需要引入栈？它在中央处理器中起到了什么样的作用？

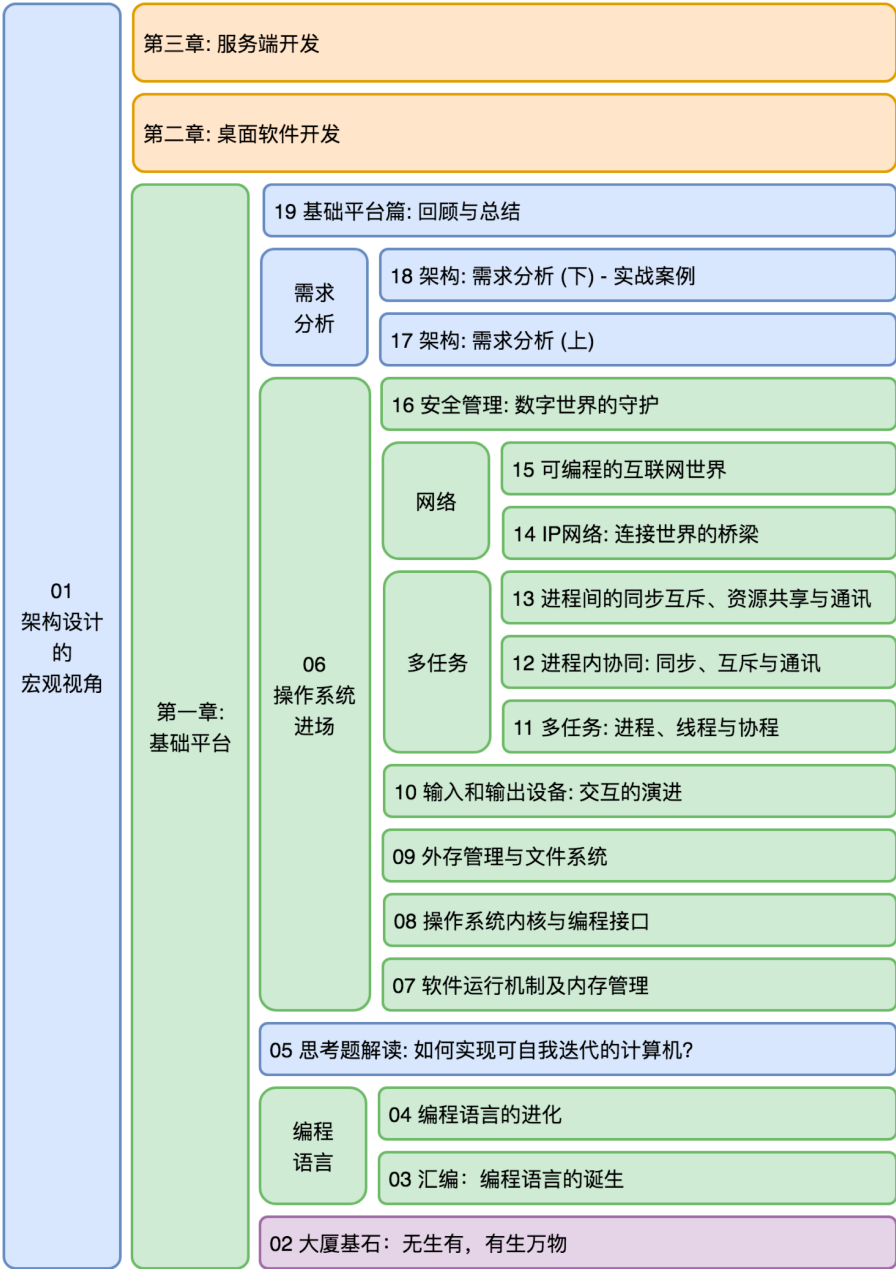
要了解这个问题，你就需要深入到中央处理器的架构设计中去。如果你对梳理中央处理器的架构设计感兴趣，可以尝试写一篇介绍它的文字。

做这样的事情会对你非常的锻炼。“**你自己理解一个事物**”和“**把你的理解表述成文，去引导其他人也能够理解它**”，是完全不同难度的事情。

如果你对中央处理器的设计细节感兴趣，可以进一步查阅相关的参考资料。也欢迎与我分享你的心得体会。

基础平台篇的内容回顾

这一章前面我们讲了些什么？为了让大家对第一章内容有个宏观的了解，我画了一幅图，如下。



首先，我们介绍了冯·诺依曼体系结构。从需求演进角度看，虽然我们信息科技发展日新月异，但是底层设计并没有发生过变化，非常稳定。从这一点来说，我们不能不佩服他们的远见。

随后，我们介绍了编程语言的演进。从汇编语言的诞生，出现了程序员这个新职业开始，此后编程语言的演进便进入高速发展期。

然而，尽管语言很多，但是编程范式的演进却并不剧烈。大家熟知的过程式、函数式、面向对象基本上能够把几乎所有的语言都囊括其中。Go 语言独树一帜地宣称自己是面向连接的语言，**我们着重对比了面向对象与面向连接思想上的差异。**

编程语言本身与业务架构的设计关联性不大，虽然模块规格的描述会借助语言的文法。**但是语言长期演进所沉淀下来的社区资源，是我们架构设计所依赖的重要基础。**充分利用好这些资源可以大大降低系统的研发成本。

最后，我们开始聊操作系统。从 UNIX => DOS => Windows/Mac/Linux => iOS/Android，从用户交互、进程管理、安全管理等角度看，操作系统的需求演变非常剧烈。

传统操作系统主要包含五个子系统：设备管理（包括存储设备、输入 / 输出设备、网络设备）、进程管理和安全管理。

输入 / 输出设备主要和交互有关，我们概要描述，基本上一笔带过。我会在后面 “桌面软件开发” 这一章再详加讨论。而服务端的交互比较简单，命令行基本上就满足需求，所以 “服务端开发” 一章我们不会再特意去展开。

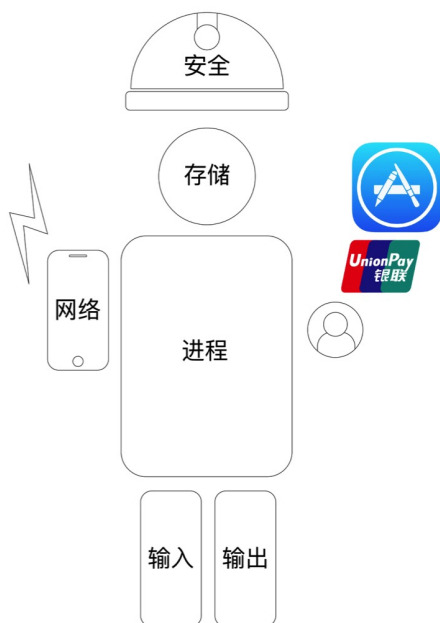
另外，操作系统的商业模式也发生了剧烈的变化。

早期操作系统的营收模式以软件销售收入为主。但是从苹果的 iOS 开始，操作系统都无一例外地增加了以下三个模块：

账号 (Account) ；

支付 (Pay) ；

应用市场 (AppStore) 。



注意，这里我们说的账号是指互联网账号。传统操作系统虽然也有账号概念，但是，它是本地账号，属于多用户权限隔离所需。

而互联网账号的价值完全不同，它是支付和应用商店的基础。没有账号，就没有支付系统，也没有办法判断用户是否在应用市场上购买过软件。

实现了“帐号 - 支付 - 应用市场”这样的商业闭环，意味着操作系统的商业模式，从软件销售转向了收税模式。这类操作系统，我们称之为现代操作系统。所有现代操作系统，所凭借的都是自己拥有巨大的流量红利。

基础平台篇的参考资料

概要回顾了我们“基础平台篇”的内容后，我们这里补充一下有助于理解我们内容的相关资料，如下。

类别	资料	作者	备注
计算机组成原理	计算机组成与设计：硬件/软件接口	David A. Patterson John L. Hennessy	计算机组成与设计的经典畅销教材。
	计算机组成：结构化方法	Andrew S.Tanenbaum Todd Austin	采用结构化方法介绍计算机系统，秉承“计算机是由层次结构组成的，每层完成规定的功能”。
	计算机是怎样跑起来的	矢泽久雄	尝试用最浅显的语言来介绍计算机原理。
	专栏: 深入浅出计算机组成原理	徐文浩	比较切实做到了深入浅出，推荐。
编程语言	C程序设计语言	Brian W. Kernighan Dennis M.Rit chie	传世经典。
	深度探索C++对象模型	Stanley B.Lippman	个人已经不推荐C++编程了，但是如果想研究下，推荐这本。
	Java编程思想	Bruce Eckel	Java语言传世经典。
	Go程序设计语言	Brian W. Kernighan Alan Donovan	传世经典《C程序设计语言》作者再次动笔。
	Go语言学习笔记	雨痕	不止于使用，也对运行时源码做出深度剖析。
	Go语言编程	七牛云团队	国内第一本 Go 书，近期会出第二版修订版，更新所有的语言变化。
	程序是怎样跑起来的	矢泽久雄	尝试用最浅显的语言来介绍软件背后的执行原理。
	专栏: Go语言核心36讲	郝林	极客时间唯一Go专栏，不二之选。
	视频课程: Go语言从入门到实战	蔡超	极客时间唯一Go视频教程。
操作系统	操作系统概念	Abraham Silberschatz Peter B. Galvin Greg Gagne	操作系统导论课程的经典书籍，被国内外众多高校选作教材。
	现代操作系统	Andrew S. Tanenbaum Herbert Bos	操作系统领域的经典教材，内容也比较与时俱进。
	UNIX编程艺术	Eric S.Raymond	传世经典，介绍UNIX设计和开发哲学、思想文化体系、原则与经验。

	UNIX环境高级编程	W. Richard Stevens Stephen A. Rago	被誉为UNIX编程“圣经”。
	UNIX网络编程 卷2：进程间通信	W. Richard Stevens	名为网络编程但只讨论单机的进程间通讯。
	专栏: 趣谈Linux操作系统	刘超	极客时间唯一操作系统专栏，源码剖析。
网络编程	TCP/IP详解 卷1：协议	Kevin R. Fall W. Richard Stevens	传世经典。
	TCP/IP详解 卷2：实现	Gary R. Wright W. Richard Stevens	传世经典。
	TCP/IP详解 卷3：T/TCP、HTTP、NNTP和UNIX域协议	W. Richard Stevens	T/TCP是一种试验性的协议，因为存在安全性问题并未被推广。
	UNIX网络编程 卷1：套接字API	W. Richard Stevens Bill Fenner Andrew M. Rudoff	因作者去世，这部经典著作由知名网络专家执笔修订了新版。
	计算机网络	Andrew S. Tanenbaum	专业的计算机网络经典教材。
	专栏: 趣谈网络协议	刘超	用有趣浅显的语言介绍网络协议，推荐。
	专栏: 透视HTTP协议	罗剑锋	还没有看，供参考。
	微课: 深入浅出gRPC	李林锋	还没有看，供参考。

有了本专栏梳理的骨架，相信对你学习和理解以上这些材料会一定的指引意义。

如果你有什么推荐的优秀参考资料，也欢迎在留言区分享，我补充到这个表格中来，我们一起来完善它。

架构之美在于悟

信息世界是无中生有创造出来的，我们不需要去记忆，而是要找到创造背后的骨架和逻辑。

架构即创造。

学架构在于匠心和悟心。它靠的是悟，不是记忆。**用思考的方式去记忆，而不是用记忆的方式去思考。**

我们日常所依赖的基础平台，随处可见的架构之美，**看到了，悟到了，就学到了。**如果你只能从你自己写业务代码中感受架构之道，那么你可能就要多留些心思了。

比如，如果你日常用的是 Go 语言，那么你可以做一个作业：“谈谈 Go 语言之美”。你从 Go 语言的设计中感悟到了什么样的架构思维？当然如果你不常接触 Go 语言，可以给自己换一个题目，比如“Java 语言之美”。

作为架构师，如何构建需求分析能力，尤其是需求的预判能力？

首先，归纳总结能力很重要。分析现象背后的原因，并对未来可能性进行推测。判断错了并不要紧，分析一下你的推测哪些地方漏判了，哪些重要信息没有考虑到。

另外，批判精神也同样至关重要。批判不是无中生有的批评，而是切实找到技术中存在的效率瓶颈和心智负担。尤其在你看经典书籍的时候，要善于找出现状与书的历史背景差异，总结技术演进的螺旋上升之路，培养科学的批判方法论。

结语

今天我们对本章内容做了概要的回顾，并借此对整个基础平台的骨架进行了一次梳理。

我们最为依赖，也最为强调的，是抽象能力。它对于构建信息世界的骨架至关重要。为此我们需要不断改造自己的抽象体系。例如，前面 [“02 | 大厦基石：无生有，有生万物”](#) 这一讲中提到过：

引入了输入输出设备的电脑，不再只能做狭义上的“计算”（也就是数学意义上的计算），如果我们把交互能力也看做一种计算能力的话，电脑理论上能够解决的“计算”问题变得无所不包。

有同学留言问：输入 / 输出设备提供的明明是一种 IO 能力，怎么能够算得上是“计算”？

但是实际上，我们人类其实就是在这种“否定自己，不断延展自己的抽象体系”，补全自己的想象力。我们以数学中最为基础的“数”为例子。数的演化大概经历了：

自然数 => 整数 => 有理数 => 实数 => 复数

输入 / 输出能力算不算是“计算”？我们不妨以广义的“计算”角度来看。

输入 (Input)，无非是采集物理世界的信息，将其数字化，所以一个输入设备其实可以看作是一个模数转换的“算子”。只不过这个算子非 CPU 的指令可以表达。

输出 (Output)，无非是将数字内容反作用于物理世界，一个输出设备其实可以看作是一个数模转换的“算子”。同样，这个算子非 CPU 的指令可以表达。

计算机 CPU 自身只能做数数转换，输入是比特信息，输出还是比特信息。结合了输入 / 输出设备提供的数模和模数转换的“算子”，连接了数字世界和物理世界的计算机，在数学上也就完备了。

如果你对今天的内容有什么思考与解读，欢迎给我留言，我们一起讨论。本章到此结束，我们将开始第二章：桌面开发的宏观视角。

如果你觉得有所收获，也欢迎把文章分享给你的朋友。感谢你的收听，我们下期再见。

 极客时间

许式伟的架构课

从源头出发，带你重新理解架构设计

许式伟
七牛云 CEO



新版升级：点击「 请朋友读」，20位好友免费读，邀请订阅更有**现金**奖励。

限时放送

推荐阅读专栏《Go 语言核心 36 讲》正在拼团中，限时特惠 79 元，[点击链接](#)订阅专栏。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 18 | 架构：需求分析 (下) · 实战案例

下一篇 加餐 | 我看Facebook发币（上）：区块链、比特币与L...

精选留言 (19)

写留言



Chen

2019-06-18

作为一个培训出来的程序员。认真拜读了基础篇，每一章都有醍醐灌顶的感觉。感谢许老师的分享



7

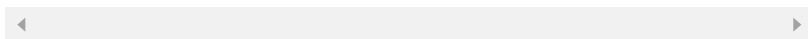


Geek_38012c6589d3

2019-06-18

可不可以讲下为什么不推荐c++了？

作者回复: 太复杂



4



MJ

2019-06-18

大学有一个专业，叫计算机科学与技术，技术是学出来的，科学是悟出来的。



👍 3

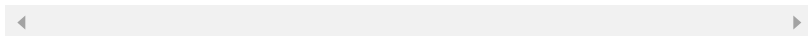


张初炼

2019-06-20

计算机组成原理（计算机体系结构）的“圣经”：计算机体系结构•量化研究方法。老师是否考虑把这本书加到参考资料里面？

作者回复: 多谢推荐



👍 1



随心而至

2019-06-20

深入理解计算机系统，我觉得挺好，边学边做题。

另外，很喜欢老师这种自底向上的教授方式，一眼就可以看出自己知识的盲点。



👍 1

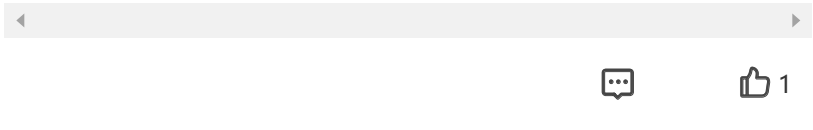


诗泽

2019-06-18

许老师，人工智能被认为是下一波技术浪潮，您认为随着人工智能的发展它会架构的设计带来什么样的变化，架构师和做工程的同学应该应对这些变化呢？谢谢！

作者回复: 不用对人工智能的影响过于放大，我们可能会以加餐的方式谈一些热门技术话题的看法



Geek_bc86bb

2019-06-18

我对基础平台的各个组件都有些许了解，但无法串联起来。而许老师用基础平台实例讲解架构需求，角度很新颖，立意很大，非常有助于将分散的知识点连接起来。课程很好很强大。



Aaron Cheung

2019-06-18

打卡19 fight



荆仙

2019-06-21

学了老师课真正感受到了架构的力量和美，真是处处皆架构，连课程目录也有如此良好的架构设计



苟范儿

2019-06-20

好多经典书籍～

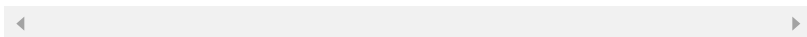


靠人品去赢

2019-06-19

大佬，作为一个技术不高的相关专业毕业的Java（也做点前端）的程序员，需不需要回头再把C捡起来，上学的时候竟顾着60分万岁了。

作者回复: 想做就去做



Dimple

2019-06-19

技术在精不在多，所以我一如既往地现在Java精进的道路上走着，回头可以好好试试看，尝试下“Java 语言之美”。

在完成Java的基础上，再去感受更高的架构之美。



honnkyou

2019-06-18

老师，刘超老师的趣谈Linux有些啃不动。要怎么学好(学会)那么课，您有什么好的建议吗？

作者回复: 我试读过，觉得门槛的确比较高。建议先学习linux编程，而不是去读源码



honnkyou

2019-06-18

扩大“计算”含义的边界就连通了数字虚拟世界与物理世界。震惊到了。^_^



川杰

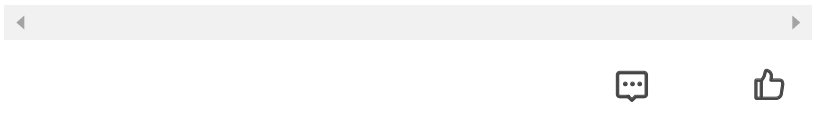
2019-06-18

您好，我看了您的PPT：GO，Next C；有几个问题想请教下：

1、非侵入式接口。我其实完全不理解这个设计好在哪里，我甚至认为这是十分糟糕的设计。比如，当我在阅读代码的时候，想要知道这个类实现了哪些接口，我很难...

作者回复: 1、这个问题我在七牛云团队写的《Go语言编程》序言中有讲。简单说，接口继承最大的问题是搞错了接口的主体。其实接口是组件的使用方定义的，而不是组件的实现方定义的。这是根源。

2、单一职责和高内聚，那都是OOP使用者的事情，不是OOP的。作为产品，OOP提供的核心能力是类和类方法（成员函数）、接口（多态）、封装（Go提供了极简的访问权限控制）。

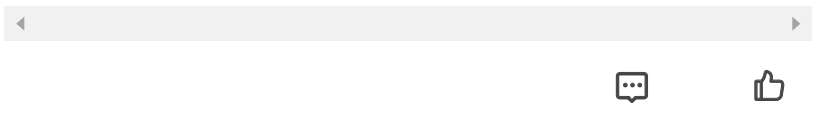


清歌

2019-06-18

请问一下，如果现在要做服务器端的开发，就必须学go语言了？

作者回复: 当然不是必须的，条条大路通罗马



觉



2019-06-18

一门深入 长时薰修



多多洛

2019-06-18

站在巨人的肩膀看架构，原来架构是如此之美



黎

2019-06-18

受益匪浅

